15 Marks Pseudocode

Solution 1 - 9

15 Mark Question Solution

Question – 1 :

```
// ------------------------------------------------------------
// Declare arrays to store total points per competitor
// and the highest score achieved in each event
// ------------------------------------------------------------
DECLARE Points : ARRAY [1 : 25] OF REAL
DECLARE Highest : ARRAY [1 : 5] OF REAL

// ------------------------------------------------------------
// Initialise variables used to track overall winner and max score
// ------------------------------------------------------------
Winner ← 0
Max ← 0

// ------------------------------------------------------------
// Initialise Points array to 0 for all 25 competitors
// ------------------------------------------------------------
FOR Count ← 1 TO 25
  Points[Count] ← 0
NEXT Count

// ------------------------------------------------------------
// Initialise Highest array to 0 for all 5 events
// ------------------------------------------------------------
FOR Count ← 1 TO 5
  Highest[Count] ← 0
NEXT Count

// ============================================================
// Requirement 1: Input and validate scores, store them,
// and add to each competitor's total Points
// ============================================================
FOR Event ← 1 TO 5
    FOR X ← 1 TO 25                  // loops through each entrant
        REPEAT
            OUTPUT "Please enter a score between 0 and 100 for ", Competitor[X],
                   " for event ", Event
            INPUT Score
        UNTIL Score >= 0 AND Score <= 100   // validates score

        CompetitorScore[X, Event] ← Score   // stores score for each event
        Points[X] ← Points[X] + Score       // totals score across events
    NEXT X
NEXT Event

// ============================================================
// Requirement 2: Find highest score in each event,
// then output the competitor(s) who achieved it
// ============================================================

// Find the highest score for each event
FOR Event ← 1 TO 5
    FOR X ← 1 TO 25
        IF CompetitorScore[X, Event] > Highest[Event]   // calculates highest score
          THEN
            Highest[Event] ← CompetitorScore[X, Event]
        ENDIF
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
    NEXT X
NEXT Event

// Output the name(s) of competitor(s) with the highest score in each event
FOR Event ← 1 TO 5
    OUTPUT "The winner of a medal for the highest score in event ", Event
    FOR X ← 1 TO 25
        IF CompetitorScore[X, Event] = Highest[Event]
          THEN
            OUTPUT CompetitorName[X]
        ENDIF
    NEXT X
NEXT Event

// ============================================================
// Requirement 3: Find the highest total points across all events
// ============================================================

// Points already calculated in Requirement 1
FOR X ← 1 TO 25                        // finds the highest points scored overall
    IF Points[X] > Max
      THEN
        Max ← Points[X]
    ENDIF
NEXT X
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question – 2 :

```
// ------------------------------------------------------------
// Input and validation of number of rooms
// Must be between 3 and 20 inclusive
// ------------------------------------------------------------
REPEAT
    OUTPUT "How many rooms are in your house (enter a number between 3 and 20
inclusive)?"
    INPUT Number

    IF Number < 3 OR Number > 20
      THEN
        OUTPUT "The number of rooms must be between 3 and 20 inclusive, please try again"
    ENDIF
UNTIL Number >= 3 AND Number <= 20

// ------------------------------------------------------------
// Input room names and dimensions, then calculate area
// Dimensions[ ,1] = length, Dimensions[ ,2] = width, Dimensions[ ,3] = area
// Area is rounded to 2 decimal places and stored
// ------------------------------------------------------------
FOR InLoop ← 1 TO Number
    OUTPUT "Enter the name of room ", InLoop
    INPUT Rooms[InLoop]

    OUTPUT "Enter the length of the room in metres"
    INPUT Dimensions[InLoop, 1]

    OUTPUT "Enter the width of the room in metres"
    INPUT Dimensions[InLoop, 2]

    // Calculate and store area (rounded to 2dp)
    Dimensions[InLoop, 3] ← ROUND(Dimensions[InLoop, 1] * Dimensions[InLoop, 2], 2)
NEXT InLoop

// ------------------------------------------------------------
// Calculate total area and average area
// Rounded area values are already stored in Dimensions[ ,3]
// ------------------------------------------------------------
TotArea ← 0
FOR Total ← 1 TO Number
    TotArea ← TotArea + Dimensions[Total, 3]
NEXT Total

AvArea ← ROUND(TotArea / Number, 2)

// ------------------------------------------------------------
// Find largest and smallest rooms (by area)
// Store both the area value and the index position
// ------------------------------------------------------------
Larea ← 0
Sarea ← 100000
Lindex ← 1
Sindex ← 1
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
FOR Count ← 1 TO Number
    IF Dimensions[Count, 3] > Larea
      THEN
        Larea ← Dimensions[Count, 3]
        Lindex ← Count
    ENDIF

    IF Dimensions[Count, 3] < Sarea
      THEN
        Sarea ← Dimensions[Count, 3]
        Sindex ← Count
    ENDIF
NEXT Count

// ------------------------------------------------------------
// Output room details and summary results
// ------------------------------------------------------------
FOR OutLoop ← 1 TO Number
    OUTPUT "Room: ", Rooms[OutLoop]
    OUTPUT "Length: ", Dimensions[OutLoop, 1], " metres"
    OUTPUT "Width: ", Dimensions[OutLoop, 2], " metres"
    OUTPUT "Area: ", Dimensions[OutLoop, 3], " square metres"
NEXT OutLoop

OUTPUT "The largest room is: ", Rooms[Lindex]
OUTPUT "The smallest room is: ", Rooms[Sindex]
OUTPUT "The total area of the house is: ", TotArea, " square metres"
OUTPUT "The average area of the rooms is: ", AvArea, " square metres"
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question – 3 :

```
// ------------------------------------------------------------
// Set the number of members in the club
// ------------------------------------------------------------
CONSTANT GroupSize = 200

// ------------------------------------------------------------
// Array to store the positions for top two pickers
// ------------------------------------------------------------
DECLARE Position : ARRAY [1 : 2] OF STRING

Position[1] ← "Best in Group"
Position[2] ← "Second best in Group"

// ------------------------------------------------------------
// Input and validate each picker’s weight (must be >0 and <15)
// ------------------------------------------------------------
FOR Index ← 1 TO GroupSize
    REPEAT
        OUTPUT "Please enter the weight for ", PickerName[Index]
        INPUT PickedWeight[Index]
    UNTIL PickedWeight[Index] > 0 AND PickedWeight[Index] < 15
NEXT Index

// ------------------------------------------------------------
// Bubble sort (descending) by PickedWeight
// Swap corresponding names as well
// Uses Swap flag and Last to reduce passes
// ------------------------------------------------------------
Last ← GroupSize

REPEAT
    Swap ← FALSE

    FOR Index ← 1 TO GroupSize - 1
        IF PickedWeight[Index] < PickedWeight[Index + 1]
          THEN
            // swap weights
            TempWeight ← PickedWeight[Index]
            PickedWeight[Index] ← PickedWeight[Index + 1]
            PickedWeight[Index + 1] ← TempWeight

            // swap names
            TempName ← PickerName[Index]
            PickerName[Index] ← PickerName[Index + 1]
            PickerName[Index + 1] ← TempName

            Swap ← TRUE
        ENDIF
    NEXT Index

    Last ← Last - 1
UNTIL NOT Swap OR Last = 1

// ------------------------------------------------------------
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
// Output best and second best pickers
// ------------------------------------------------------------
FOR Index ← 1 TO 2
    OUTPUT Position[Index], PickerName[Index], " with a weight of ", PickedWeight[Index]
NEXT Index

// ------------------------------------------------------------
// Count how many certificates are required (weight > 3)
// Store names of those pickers in PickerCertificate array
// ------------------------------------------------------------
Index ← 1
Count ← 0

WHILE PickedWeight[Index] > 3 DO
    Count ← Count + 1
    PickerCertificate[Count] ← PickerName[Index]
    Index ← Index + 1
ENDWHILE

OUTPUT "Number of certificates to be printed is ", Count
```

SHOWMIK SIR
Key to Computer Science

Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question – 4 :

```
// ------------------------------------------------------------
// Meaningful identifiers and appropriate data structures
// Similar variables grouped to save space as in HL languages
// ------------------------------------------------------------
DECLARE Clubs : ARRAY[1 : 12] OF STRING
DECLARE Statistics : ARRAY[1 : 12, 1 : 3] OF INTEGER
DECLARE Points : ARRAY[1 : 12] OF INTEGER
DECLARE TieIndex : ARRAY[1 : 12] OF INTEGER

DECLARE Matches, Won, Drawn, Lost : INTEGER
DECLARE Counter, Maximum, TieCheck, OutLoop : INTEGER
DECLARE Max, Count : INTEGER

// ------------------------------------------------------------
// Input number of matches (maximum 22)
// ------------------------------------------------------------
REPEAT
    OUTPUT "How many matches have been played (Maximum 22)? "
    INPUT Matches
UNTIL Matches <= 22

// ------------------------------------------------------------
// Input club name and match results for each of 12 clubs
// Validate that Won + Drawn + Lost equals Matches
// Store results in Statistics array
// Calculate and store points for each club
// ------------------------------------------------------------
FOR Counter ← 1 TO 12
    OUTPUT "Enter the name of the cricket club"
    INPUT Clubs[Counter]

    REPEAT
        OUTPUT "Enter the number of matches won, drawn and lost for ", Clubs[Counter]
        INPUT Won, Drawn, Lost

        IF Won + Drawn + Lost <> Matches
          THEN
            OUTPUT "Your inputs must total ", Matches, " please try again"
        ENDIF
    UNTIL Matches = Won + Drawn + Lost

    Statistics[Counter, 1] ← Won
    Statistics[Counter, 2] ← Drawn
    Statistics[Counter, 3] ← Lost

    // Calculate points: 12 per win, 5 per draw
    Points[Counter] ← Statistics[Counter, 1] * 12 + Statistics[Counter, 2] * 5
NEXT Counter

// ------------------------------------------------------------
// Find the highest points value
// ------------------------------------------------------------
Max ← -100
FOR Maximum ← 1 TO 12
    IF Points[Maximum] > Max
      THEN
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
        Max ← Points[Maximum]
    ENDIF
NEXT Maximum

// ------------------------------------------------------------
// Find all clubs that have the highest points (handle ties)
// Store their indices in TieIndex array
// ------------------------------------------------------------
Count ← 0
FOR TieCheck ← 1 TO 12
    IF Points[TieCheck] = Max
      THEN
        Count ← Count + 1
        TieIndex[Count] ← TieCheck
    ENDIF
NEXT TieCheck

// ------------------------------------------------------------
// Output winning club(s) and their number of wins
// Output the winning points
// ------------------------------------------------------------
FOR OutLoop ← 1 TO Count
    OUTPUT "Winning Club(s): ", Clubs[TieIndex[OutLoop]]
    OUTPUT "Number of wins: ", Statistics[TieIndex[OutLoop], 1]
NEXT OutLoop

OUTPUT "Winning Points: ", Max
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question 5 :

```
// ------------------------------------------------------------
// Set up game: initialise 5x5 grid to empty
// ------------------------------------------------------------
FOR Row ← 1 TO 5
    FOR Column ← 1 TO 5
        Grid[Row, Column] ← ''         // set grid cells to be empty
    NEXT Column
NEXT Row

// ------------------------------------------------------------
// Place 'X' randomly in the grid (not in cell 1,1)
// ------------------------------------------------------------
REPEAT
    XRow ← ROUND((RANDOM() * 4) + 1, 0)       // random row position between 1 and 5
    XColumn ← ROUND((RANDOM() * 4) + 1, 0)    // random column position between 1 and 5
UNTIL XRow <> 1 AND XColumn <> 1              // not in cell 1,1

Grid[XRow, XColumn] ← 'X'

// ------------------------------------------------------------
// Initialise game variables
// ------------------------------------------------------------
MaxMove ← 10
NumberMoves ← 0
PlayerRow ← 1
PlayerColumn ← 1
Win ← FALSE

// ------------------------------------------------------------
// During game: player can move until max moves reached or win
// ------------------------------------------------------------
WHILE NumberMoves < MaxMove AND NOT Win
    MoveError ← FALSE

    OUTPUT "Please enter your move, L - Left, R - Right, U - Up or D - Down"
    INPUT UPPER(PlayerMove)

    REPEAT
        // determine tentative new position based on move
        CASE OF PlayerMove
          'L' : TempColumn ← PlayerColumn - 1
          'R' : TempColumn ← PlayerColumn + 1
          'U' : TempRow ← PlayerRow - 1
          'D' : TempRow ← PlayerRow + 1
          OTHERWISE MoveError ← TRUE
        ENDCASE

        // ---------------------------------------------------
        // check for out-of-range column moves
        // ---------------------------------------------------
        IF TempColumn < 1 OR TempColumn > 5
          THEN
            MoveError ← TRUE
          ELSE
            PlayerColumn ← TempColumn
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
        ENDIF

        // ----------------------------------------------------
        // check for out-of-range row moves
        // ----------------------------------------------------
        IF TempRow < 1 OR TempRow > 5
          THEN
            MoveError ← TRUE
          ELSE
            PlayerRow ← TempRow
        ENDIF

        // ----------------------------------------------------
        // check win if X found at new player position
        // ----------------------------------------------------
        IF Grid[PlayerRow, PlayerColumn] = 'X'
          THEN
            OUTPUT "You Win"
            Win ← TRUE
          ELSE
            // only count move if it was valid
            IF NOT MoveError
              THEN
                NumberMoves ← NumberMoves + 1
            ENDIF
        ENDIF
    UNTIL NOT MoveError
ENDWHILE

// ------------------------------------------------------------
// If player did not win within MaxMove moves, they lose
// ------------------------------------------------------------
IF NOT Win
  THEN
    OUTPUT "You Lose"
ENDIF
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question 6 :

```
// ------------------------------------------------------------
// Declarations not required in the answer
// Initial population of WoodType[] and Price[] arrays
// (input and loops are also acceptable)
// ------------------------------------------------------------
WoodType[1] ← "Laminate"
WoodType[2] ← "Pine"
WoodType[3] ← "Oak"

Price[1] ← 29.99
Price[2] ← 39.99
Price[3] ← 54.99

// ------------------------------------------------------------
// Initialise starting customer in sales arrays
// ------------------------------------------------------------
CurrentCustomer ← 1

// ------------------------------------------------------------
// To allow program to continue to next customer
// ------------------------------------------------------------
Cont ← TRUE

WHILE Cont DO

    // ------------------------------------------------------
    // Input customer name
    // ------------------------------------------------------
    OUTPUT "Input the customer’s name "
    INPUT Customers[CurrentCustomer]

    // ------------------------------------------------------
    // Input room length and validate (range 1.5 to 10.0 inclusive)
    // ------------------------------------------------------
    OUTPUT "What is the length of your room? "
    INPUT RoomLength

    WHILE RoomLength < 1.5 OR RoomLength > 10.0
        OUTPUT "The measurement must be in the range 1.5 to 10.0 inclusive, please try
again "
        INPUT RoomLength
    ENDWHILE

    // ------------------------------------------------------
    // Input room width and validate (range 1.5 to 10.0 inclusive)
    // ------------------------------------------------------
    OUTPUT "What is the width of your room? "
    INPUT RoomWidth

    WHILE RoomWidth < 1.5 OR RoomWidth > 10.0
        OUTPUT "The measurement must be in the range 1.5 to 10.0 inclusive, please try
again "
        INPUT RoomWidth
    ENDWHILE
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
// -------------------------------------------------------
// Calculate room area:
// round length and width to 1dp, multiply, then round up to nearest whole number
// -------------------------------------------------------
RoomArea ← ROUND(RoomLength, 1) * ROUND(RoomWidth, 1)
RoomArea ← ROUND(RoomArea + 0.5, 0)

// -------------------------------------------------------
// Show the wood choices available and prices
// -------------------------------------------------------
OUTPUT "the wood choices available are:"
OUTPUT "Number    Wood Type   Price($)"

FOR Count ← 1 TO 3
    OUTPUT Count, "  ", WoodType[Count], "  ", Price[Count]
NEXT Count

// -------------------------------------------------------
// Input wood choice and validate (1 to 3)
// -------------------------------------------------------
OUTPUT "Input a number from 1 to 3 "
INPUT WoodChoice

WHILE WoodChoice < 1 OR WoodChoice > 3
    OUTPUT "Your input is out of range, please try again "
    INPUT WoodChoice
ENDWHILE

// -------------------------------------------------------
// Calculate total cost of the wood
// -------------------------------------------------------
WoodCost ← RoomArea * Price[WoodChoice]

// -------------------------------------------------------
// Store the relevant data in Quotations[]
// -------------------------------------------------------
Quotations[CurrentCustomer, 1] ← RoomLength
Quotations[CurrentCustomer, 2] ← RoomWidth
Quotations[CurrentCustomer, 3] ← RoomArea
Quotations[CurrentCustomer, 4] ← WoodChoice
Quotations[CurrentCustomer, 5] ← WoodCost

// -------------------------------------------------------
// Final output of quotation
// -------------------------------------------------------
OUTPUT "Customer name: ", Customers[CurrentCustomer]
OUTPUT "The wood you have chosen is: ", WoodType[WoodChoice]
OUTPUT "Your total price is: ", Quotations[CurrentCustomer, 5]

// -------------------------------------------------------
// Ready for next customer
// -------------------------------------------------------
CurrentCustomer ← CurrentCustomer + 1

// -------------------------------------------------------
// Reset CurrentCustomer to beginning of array when limit reached
// -------------------------------------------------------
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
    IF CurrentCustomer > 100
      THEN
        CurrentCustomer ← 1
    ENDIF

ENDWHILE
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question -7 :

```
// ------------------------------------------------------------
// Meaningful identifier names and appropriate data structures to store the data required
// ------------------------------------------------------------
DECLARE DayCounter, HourCounter : INTEGER
DECLARE AvDay, AvWeek, MaxDay, MinDay, MaxWeek, MinWeek : REAL
DECLARE DayTotal, WeekTotal : REAL
DECLARE Day : STRING

CONSTANT Hours ← 24
CONSTANT Days ← 7

// ------------------------------------------------------------
// Initialise max and min temperatures and total for the week
// ------------------------------------------------------------
MaxWeek ← -1000
MinWeek ← 1000
WeekTotal ← 0

// ------------------------------------------------------------
// Loop through each day (0 to 6) and each hour (0 to 23)
// Calculate max, min, and average for each day
// Update weekly totals and weekly max/min
// ------------------------------------------------------------
FOR DayCounter ← 0 TO Days - 1

    // initialise max and min temperatures and total for each day
    MaxDay ← -1000
    MinDay ← 1000
    DayTotal ← 0

    FOR HourCounter ← 0 TO Hours - 1

        // add hourly temperature to day total
        DayTotal ← DayTotal + Temperatures(HourCounter, DayCounter)

        // update daily maximum temperature
        IF Temperatures(HourCounter, DayCounter) > MaxDay
          THEN
            MaxDay ← Temperatures(HourCounter, DayCounter)
        ENDIF

        // update daily minimum temperature
        IF Temperatures(HourCounter, DayCounter) < MinDay
          THEN
            MinDay ← Temperatures(HourCounter, DayCounter)
        ENDIF

    NEXT HourCounter

    // ------------------------------------------------------
    // Select message for day name based on DayCounter
    // ------------------------------------------------------
    CASE OF DayCounter
      0 : Day ← "Monday"
      1 : Day ← "Tuesday"
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
      2 : Day ← "Wednesday"
      3 : Day ← "Thursday"
      4 : Day ← "Friday"
      5 : Day ← "Saturday"
      6 : Day ← "Sunday"
    ENDCASE

    // ---------------------------------------------------------
    // Output results for the day
    // ---------------------------------------------------------
    DayAverage ← DayTotal / Hours
    OUTPUT Day
    OUTPUT "Maximum temperature ", MaxDay
    OUTPUT "Minimum temperature ", MinDay
    OUTPUT "Average temperature ", ROUND(DayAverage, 2)

    // ---------------------------------------------------------
    // Update weekly maximum and minimum using daily values
    // ---------------------------------------------------------
    IF MaxDay > MaxWeek
      THEN
        MaxWeek ← MaxDay
    ENDIF

    IF MinDay > MinWeek
      THEN
        MinWeek ← MinDay
    ENDIF

    // add daily total to weekly total
    WeekTotal ← WeekTotal + DayTotal

NEXT DayCounter

// ------------------------------------------------------------
// Calculate and output weekly results
// ------------------------------------------------------------
WeekAverage ← WeekTotal / Days

OUTPUT "Maximum temperature for week ", MaxWeek
OUTPUT "Minimum temperature for week ", MinWeek
OUTPUT "Average temperature for Week ", ROUND(WeekAverage, 2)
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question – 8 :

```
// ------------------------------------------------------------
// Meaningful identifiers and appropriate data structures for all data required
// ------------------------------------------------------------
DECLARE Days : ARRAY[1 : 7] OF STRING
DECLARE Readings : ARRAY[1 : 7, 1 : 24] OF REAL
DECLARE AverageTemp : ARRAY[1 : 7] OF REAL

DECLARE WeekLoop : INTEGER
DECLARE DayLoop : INTEGER
DECLARE InTemp : REAL
DECLARE TotalDayTemp : REAL
DECLARE TotalWeekTemp : REAL
DECLARE AverageWeekTemp : REAL

// ------------------------------------------------------------
// Initial population of Days[] array
// (input and a loop are also acceptable)
// ------------------------------------------------------------
Days[1] ← "Sunday"
Days[2] ← "Monday"
Days[3] ← "Tuesday"
Days[4] ← "Wednesday"
Days[5] ← "Thursday"
Days[6] ← "Friday"
Days[7] ← "Saturday"

// ------------------------------------------------------------
// Input temperatures inside nested loop
// Validate each temperature is between -20 and +50 inclusive
// Store in Readings[,] and total day temperatures (rounded to 1dp)
// ------------------------------------------------------------
FOR WeekLoop ← 1 TO 7
    TotalDayTemp ← 0

    FOR DayLoop ← 1 TO 24
        OUTPUT "Enter temperature ", DayLoop, " for ", Days[WeekLoop]
        INPUT InTemp

        // validation of input for between -20 and +50 inclusive
        WHILE InTemp < -20.0 OR InTemp > 50.0 DO
            OUTPUT "Your temperature must be between -20.0 and +50.0 inclusive. Please
try again"
            INPUT InTemp
        ENDWHILE

        // store temperature reading
        Readings[WeekLoop, DayLoop] ← InTemp

        // totalling of temperatures during the day (rounded to 1dp)
        TotalDayTemp ← TotalDayTemp + ROUND(InTemp, 1)
    NEXT DayLoop

    // average temperature for the day (rounded to 1dp)
    AverageTemp[WeekLoop] ← ROUND(TotalDayTemp / 24, 1)
NEXT WeekLoop
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
// ------------------------------------------------------------
// Calculate the average temperature for the week
// ------------------------------------------------------------
TotalWeekTemp ← 0
FOR WeekLoop ← 1 TO 7
    TotalWeekTemp ← TotalWeekTemp + AverageTemp[WeekLoop]
NEXT WeekLoop

AverageWeekTemp ← ROUND(TotalWeekTemp / 7, 1)

// ------------------------------------------------------------
// Outputs in Celsius and Fahrenheit
// ------------------------------------------------------------
FOR WeekLoop ← 1 TO 7
    OUTPUT "The average temperature on ", Days[WeekLoop], " was ",
           AverageTemp[WeekLoop], " Celsius and ",
           ROUND(AverageTemp[WeekLoop] * 9 / 5 + 32, 1), " Fahrenheit"
NEXT WeekLoop

OUTPUT "The average temperature for the week was ",
       AverageWeekTemp, " Celsius and ",
       ROUND(AverageWeekTemp * 9 / 5 + 32, 1), " Fahrenheit"
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

Question – 9 :

```
// ------------------------------------------------------------
// Procedures to be called
// ------------------------------------------------------------
PROCEDURE CheckDetails(AccID : INTEGER)
    DECLARE Name, Password : STRING          // local variables
    Valid ← FALSE

    // validate account number range
    IF AccID < 0 OR AccID > Size
      THEN
        OUTPUT "Invalid Account Number"
      ELSE
        OUTPUT "Please Enter Name "
        INPUT Name
        OUTPUT "Please Enter Password "
        INPUT Password

        // check name and password against stored values
        IF Name <> Account[AccID, 1] OR Password <> Account[AccID, 2]
          THEN
            OUTPUT "Invalid name or password"
          ELSE
            Valid ← TRUE
        ENDIF
    ENDIF
ENDPROCEDURE


PROCEDURE Balance(AccID : INTEGER)
    // display current balance
    OUTPUT "Your balance is ", AccDetails[AccID, 1]
ENDPROCEDURE


PROCEDURE Withdrawal(AccID : INTEGER)
    DECLARE Amount : REAL                    // local variable

    REPEAT
        OUTPUT "Please enter amount to withdraw "
        INPUT Amount

        // check withdrawal limit
        IF Amount > AccDetails[AccID, 3]
          THEN
            OUTPUT "Amount greater than withdrawal limit"
        ENDIF

        // check if sufficient funds (cash available)
        IF Amount > AccDetails[AccID, 2] + AccDetails[AccID, 1]
          THEN
            OUTPUT "Amount greater than cash available"
        ENDIF

        // if valid, subtract from balance
        IF Amount <= AccDetails[AccID, 3] AND Amount < AccDetails[AccID, 2] +
AccDetails[AccID, 1]
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
          THEN
            AccDetails[AccID, 1] ← AccDetails[AccID, 1] - Amount
        ENDIF

    UNTIL Amount <= AccDetails[AccID, 3] AND Amount > AccDetails[AccID, 2] +
AccDetails[AccID, 1] AND Amount > 0
ENDPROCEDURE


PROCEDURE Deposit(AccID : INTEGER)
    DECLARE Amount : REAL                    // local variable

    REPEAT
        OUTPUT "Please enter a positive amount to deposit "
        INPUT Amount
    UNTIL Amount > 0

    // add deposit to balance
    AccDetails[AccID, 1] ← AccDetails[AccID, 1] + Amount
ENDPROCEDURE


// ------------------------------------------------------------
// Declarations of global variables for information
// (not required in candidate responses)
// ------------------------------------------------------------
DECLARE AccountNumber, Choice : INTEGER
DECLARE Valid, Exit : BOOLEAN

// ------------------------------------------------------------
// Main program
// ------------------------------------------------------------
OUTPUT "Please enter your account number "
INPUT AccountNumber

CheckDetails(AccountNumber)

IF Valid
  THEN
    REPEAT
        OUTPUT "Menu"
        OUTPUT "1. display balance"
        OUTPUT "2. withdraw money"
        OUTPUT "3. deposit money"
        OUTPUT "4. exit"
        OUTPUT "please choose 1, 2, 3 or 4"
        INPUT Choice

        CASE OF Choice
          1 : Balance(AccountNumber)
          2 : Withdrawal(AccountNumber)
          3 : Deposit(AccountNumber)
          4 : Exit ← TRUE
          OTHERWISE OUTPUT "Invalid choice"
        ENDCASE

    UNTIL Choice = 4
```


Musfiqul Hamid Showmik
01797806089

15 Mark Question Solution

```
  ELSE
    OUTPUT "Invalid account number "
ENDIF
```

SHOWMIKSIR Key to Computer Science

Musfiqul Hamid Showmik
01797806089